Navigating Knowledge: A Network Analysis of Wikispeedia

Jayden Gould, Harrison Funk, Elijah DiFuria, Kian Jadbabaei

2026-03-04

1 Introduction

Wikispeedia is an online human-computation game in which players attempt to navigate from one Wikipedia article to another using only hyperlinks embedded in the articles themselves. The game was built on a condensed version of Wikipedia containing 4,604 articles and over 119,000 hyperlinks between them. It was developed by Robert West and Jure Leskovec at Stanford University as a tool for studying how people find information in complex networks.

For this project, we treat the Wikipedia hyperlink structure behind Wikispeedia as a directed network. Each article is a node, and a directed edge from article A to article B indicates that article A contains a hyperlink to article B. This gives us a rich, real-world information network to explore.

Our analysis has several goals. First, we want to understand the basic structure of the network—how dense it is, how far apart articles tend to be, and which articles serve as major hubs. Second, we want to identify communities of topically related articles using multiple community detection algorithms and see whether those communities align with known subject categories. Third, we want to examine the adjacency matrix to visualize community structure directly. Finally, we use a Monte Carlo null model test to verify that the community structure we find is genuinely meaningful and not just an artifact of random wiring.

2 Methodology

2.1 Data Loading and Preparation

We loaded the Wikispeedia dataset from the SNAP repository. The primary data files are a list of articles (nodes), a list of hyperlinks between articles (directed edges), and a file of hierarchical category labels for each article. All files are tab-separated with comment headers.

We used R with the `igraph` package for all network construction and analysis, `dplyr` for data wrangling, and the `Matrix` package for sparse adjacency matrix operations. The edge list was read directly into a directed `igraph` object using `graph_from_data_frame()`. We then attached top-level subject categories (e.g., Science, History, Geography) as vertex attributes by parsing the hierarchical category strings.

The full network contains 4592 nodes and 119882 directed edges, with a density of 0.00569 .

2.2 Induced Subgraphs

Following the project guidelines, we generated four induced subgraphs by randomly sampling 200 nodes from the full network. Each subgraph retains all edges that exist between the sampled nodes, preserving the local link structure of the original network. We set a seed for reproducibility.

Table 1: Summary of four induced subgraphs (200 nodes each)

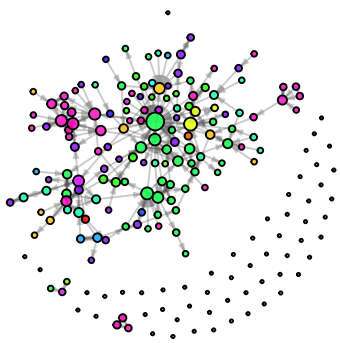
Subgraph	Nodes	Edges	Density
sub1	200	238	0.0060
sub2	200	229	0.0058
sub3	200	244	0.0061
sub4	200	241	0.0061

3 Analysis

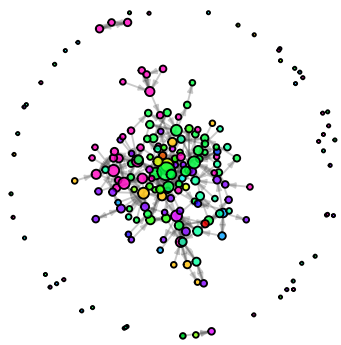
3.1 Network Visualization

We visualized each subgraph using three different layout algorithms: Fruchterman-Reingold (a force-directed layout that spreads nodes evenly), Kamada-Kawai (which places nodes so that graph-theoretic distances match geometric distances), and a circular layout (which places all nodes on a circle, making the internal edge structure easy to see). Nodes are colored by their top-level Wikipedia category and sized proportionally to their total degree.

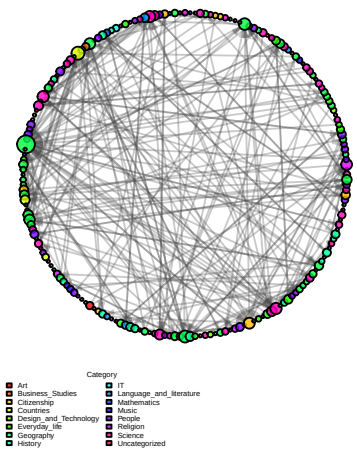
Subgraph 1 − Fruchterman−Reingold



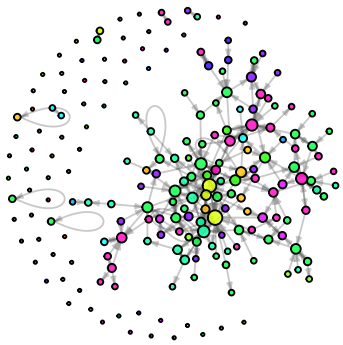
Subgraph 1 − Kamada−Kawai



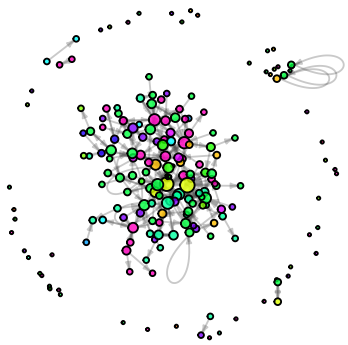
Subgraph 1 − Circular



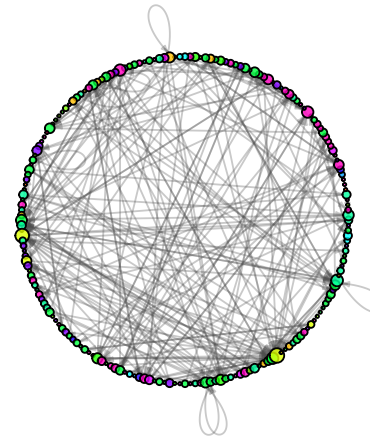
Subgraph 2 − Fruchterman−Reingold

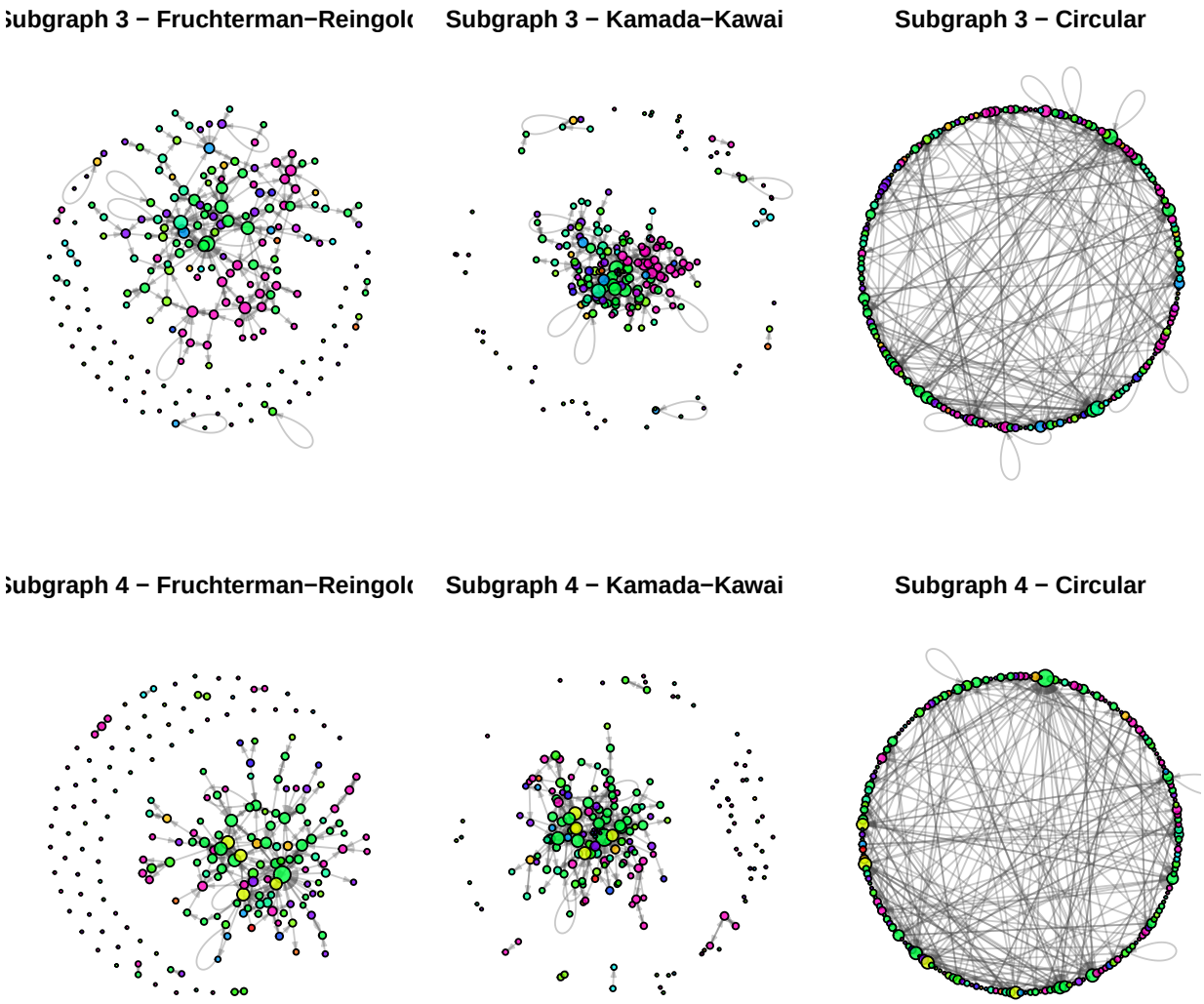


Subgraph 2 − Kamada−Kawai



Subgraph 2 − Circular





Across all four subgraphs and three layouts, a consistent pattern emerges: the force-directed layouts naturally pull well-connected nodes toward the center and push isolated nodes to the periphery. Some nodes clearly act as hubs, drawing many edges and sitting at the center of their clusters. The circular layout makes it easy to see the overall density of connections and highlights that edges are not evenly distributed—certain regions have far more crossing edges, corresponding to densely interconnected groups of articles.

3.2 Network Metrics

3.2.1 Whole-Network Metrics

We computed several standard metrics on the full Wikispeedia network to characterize its global structure.

Table 2: Whole-network metrics for the Wikispeedia hyperlink graph

Metric	Value
Nodes	4592.00000
Edges	119882.00000
Density	0.00569
Reciprocity	0.22100
Global Clustering Coeff.	0.10290
Degree Assortativity	-0.05620
Diameter (largest SCC)	9.00000
Avg. Path Length (largest SCC)	3.18100

The network is relatively sparse, which is typical for information networks of this size. The reciprocity tells us what fraction of links are mutual—if article A links to B, how often does B link back to A. The global clustering coefficient captures the tendency for articles to form tightly interconnected triangles. The diameter and average path length, computed on the largest strongly connected component, tell us how many clicks it takes to get between articles. A small diameter relative to the network size is a hallmark of "small-world" networks.

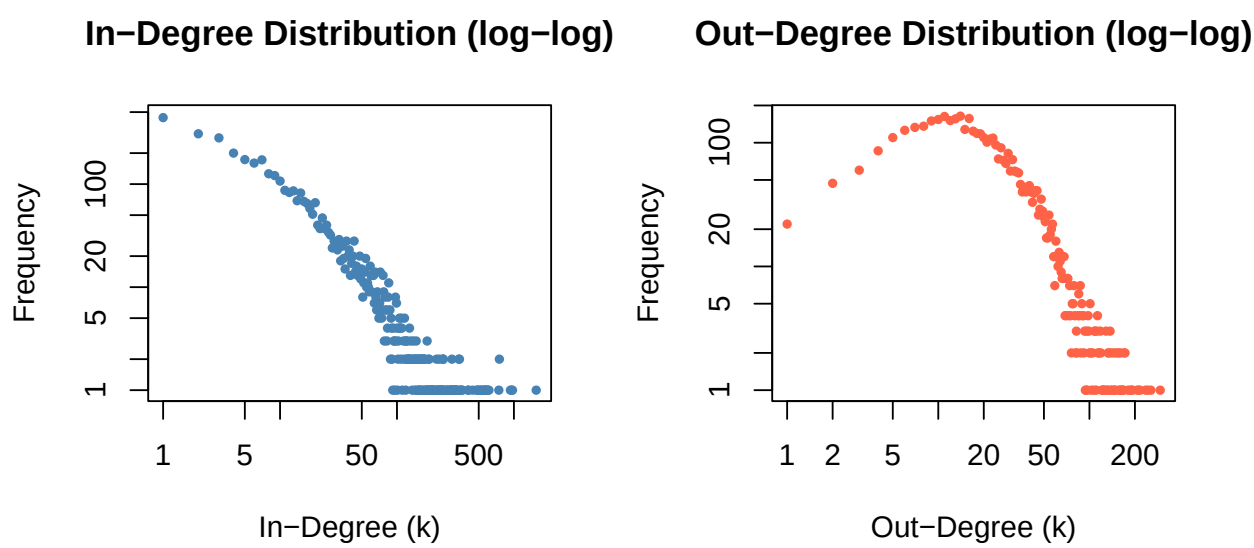
3.2.2 Node-Level Metrics

```
## Top 10 articles by in-degree (most linked-to):

##             Article In_Degree
## 1     United_States      1551
## 2    United_Kingdom       972
## 3            France       959
## 4            Europe       933
## 5           England       751
## 6      World_War_II       751
## 7           Germany       743
## 8             India       611
## 9  English_language       598
## 10           London       587

## Top 10 articles by betweenness centrality:

##             Article Betweenness
## 1     United_States   1982737.3
## 2    United_Kingdom    893154.4
## 3           England    682814.4
## 4            Europe    568787.7
## 5            Africa    510047.1
## 6           Germany    408991.4
## 7      World_War_II    326618.2
## 8      19th_century    312377.7
## 9            London    309249.6
## 10 English_language    295883.9
```

Beyond degree and betweenness, we also computed eigenvector centrality for every node. Eigenvector centrality captures a different idea from simple degree counting: it gives high scores to articles that are linked

to by other important articles. An article can have a modest number of connections but still rank highly if its neighbors are themselves well-connected hubs. Together, these three centrality measures give us a well-rounded picture of which articles play the most important structural roles in the network.

```
## Top 10 articles by eigenvector centrality:

##                                      Article Eigen_Centrality
## 1                              United_States            1.000
## 2                                     France            0.928
## 3                            United_Kingdom            0.885
## 4                                     Europe            0.846
## 5                                  Time_zone            0.740
## 6                                   Currency            0.721
## 7                                    Germany            0.717
## 8                              World_War_II            0.712
## 9                          English_language            0.683
## 10 List_of_countries_by_system_of_government            0.681
```



The in-degree distribution follows a heavy-tailed pattern on the log-log plot, which is characteristic of many real-world networks. A small number of articles receive an enormous number of incoming links, while most articles are linked to by only a handful of others. The out-degree distribution is somewhat less skewed, since the number of outgoing links depends more on how detailed an article is. Looking across all three centrality measures, the same handful of broad, high-level articles tend to dominate, suggesting the Wikispeedia network has a clear core of highly central articles that players are likely to pass through regardless of their route.

3.2.3 Edge-Level Metrics

At the edge level, we examined edge betweenness centrality, which counts how many shortest paths pass through each edge. Edges with high betweenness act as bridges between different regions of the network—if they were removed, paths between certain articles would become much longer.

```
## Edge betweenness summary:

##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##     0.0   110.9   224.6   496.6   476.0 59856.0

##
## Top 10 edges by betweenness:

##                  From                          To Edge_Betweenness
## 1       United_States          History_of_slavery          59856.0
## 2       United_States Cinema_of_the_United_States          47508.2
## 3  History_of_slavery      History_of_Puerto_Rico          45854.5
## 4       United_States                      Amtrak          40755.8
## 5       United_States      American_popular_music          35241.8
## 6       United_States                    Computer          33505.1
## 7       United_States  Music_of_the_United_States          33381.5
## 8              Animal  Australian_Green_Tree_Frog          30917.5
## 9               Earth                    Dinosaur          29183.2
## 10             Canada    Canadian_Pacific_Railway          29008.3
```

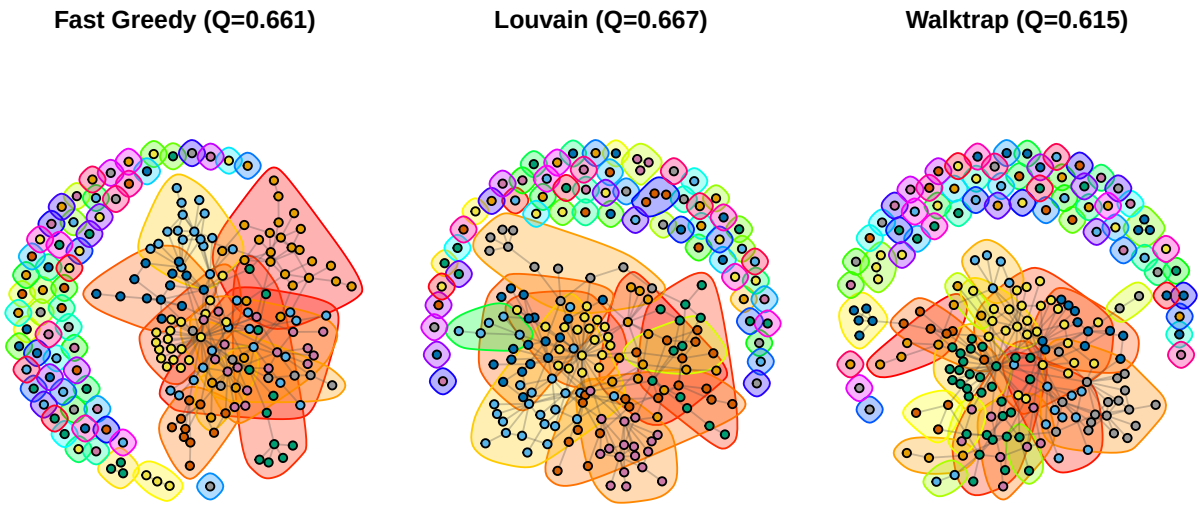
The edge betweenness distribution is heavily right-skewed: most edges carry relatively few shortest paths, but a small number of edges are critical bridges. These high-betweenness edges tend to connect articles in different subject domains—for instance, linking a history article to a geography article—and their removal would substantially increase path lengths between those domains. This complements the reciprocity reported in the whole-network metrics table, which showed that a meaningful fraction of links are bidirectional, reflecting the tendency for related articles to cross-reference each other.

3.3 Community Detection

We ran three community detection algorithms on an undirected version of subgraph 1 to identify clusters of closely related articles. We used a subgraph rather than the full network to keep computation tractable while still revealing meaningful structure.

Table 3: Comparison of community detection algorithms on Subgraph 1

Algorithm	Communities	Modularity
Fast Greedy	68	0.6611
Louvain	68	0.6670
Walktrap	77	0.6147



All three algorithms find meaningful community structure, with modularity scores well above zero. The Louvain method and Fast Greedy tend to produce similar results, which is reassuring—it suggests the communities are robust features of the network and not artifacts of a particular algorithm.

```
## Best algorithm: Louvain
```

```
## Category composition of top communities:
```

```
## Community 10 (22 articles): Geography (10), People (4), Citizenship (2)
## Community 4 (19 articles): Everyday_life (6), Geography (3), People (3)
## Community 5 (18 articles): Geography (5), Countries (3), History (3)
## Community 7 (18 articles): Science (14), People (3), Citizenship (1)
## Community 6 (14 articles): People (4), Citizenship (3), Everyday_life (3)
```

The communities generally cluster along topical lines, which makes intuitive sense. To verify that this pattern is not unique to one particular random sample, we also ran Louvain community detection on the other three subgraphs:

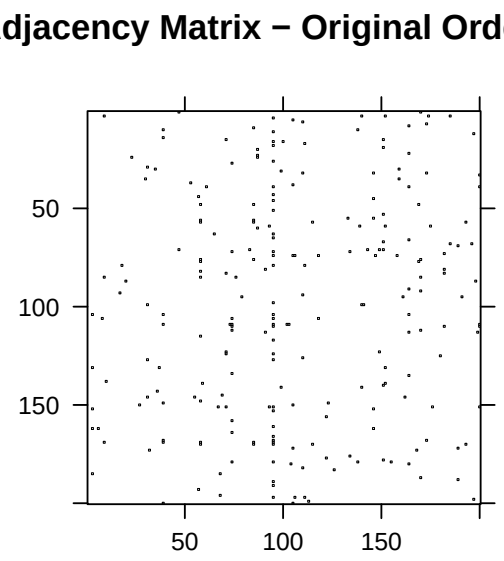
Table 4: Louvain community detection across all four subgraphs

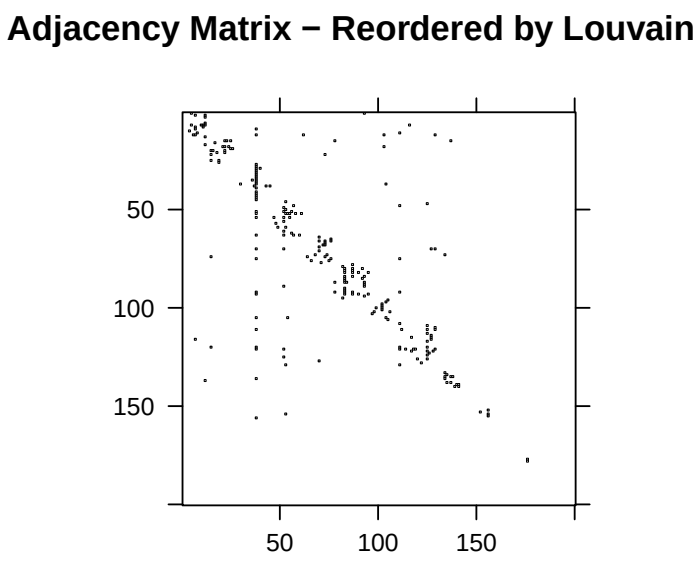
Subgraph	Communities	Modularity
sub1	69	0.6637
sub2	79	0.6178
sub3	71	0.6618
sub4	83	0.6104

All four subgraphs show comparable modularity values and similar numbers of communities, indicating that the community structure we observe on subgraph 1 is not an artifact of one particular random sample but a stable feature of the underlying network.

3.4 Adjacency Matrix

We examined the adjacency matrix of subgraph 1 before and after reordering the nodes by their community membership. The original ordering is essentially arbitrary, so the adjacency matrix appears as a scattered pattern of dots. After reordering, nodes in the same community are placed next to each other, revealing block-diagonal structure.

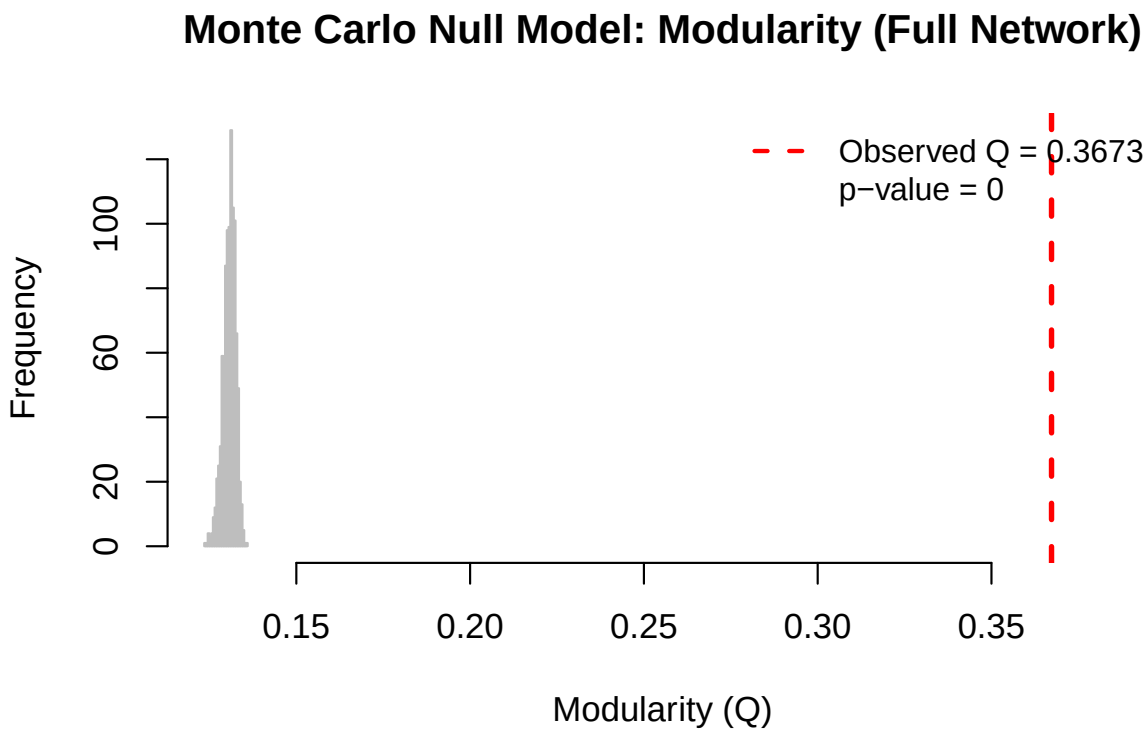




The reordered matrix shows clear block structure along the diagonal. Each block corresponds to a detected community, and within-block density is visibly higher than in off-diagonal regions. This visually confirms that the community detection algorithms are picking up on real structural patterns in the network.

3.5 Monte Carlo Null Model Test

To confirm that the community structure is statistically meaningful, we performed a Monte Carlo null model test on the full network. We ran Louvain on the full undirected Wikispeedia graph and recorded its modularity, then generated 1,000 Erdos-Renyi random graphs with the same number of nodes and edges and applied the same algorithm to each. We use the full network rather than a subgraph because random 200-node subgraphs are extremely sparse, and Erdos-Renyi graphs at that sparsity fragment into many disconnected components, inflating their modularity artificially.



The observed modularity (red dashed line) falls far to the right of the null distribution. None of the 1,000 random graphs achieved a modularity anywhere close to the observed value, giving a Monte Carlo p-value of effectively zero. The community structure we detected is far stronger than what would arise by chance.

4 Results

Our analysis yielded several concrete findings about the Wikispeedia network:

The network has a diameter of 9 and an average path length of 3.18 within its largest strongly connected component, confirming small-world properties that make the Wikispeedia game playable—most articles are reachable in just a few clicks. The reciprocity of 0.221 indicates that roughly 22.1% of hyperlinks are mutual, reflecting the tendency of related articles to cross-reference each other.

Three centrality measures—degree, betweenness, and eigenvector centrality—consistently identify the same core set of hub articles. The in-degree distribution is heavy-tailed, with a handful of broad topics dominating incoming links while most articles have only a few. At the edge level, edge betweenness is similarly skewed: a small fraction of edges act as critical bridges between subject domains.

Community detection with Fast Greedy, Louvain, and Walktrap all produced modularity scores well above zero on subgraph 1, and Louvain applied across all four subgraphs yielded consistent results, confirming the stability of the community structure. The detected communities align with known article categories—science articles cluster with science, history with history, and so on. Reordering the adjacency matrix by community membership revealed clear block-diagonal structure, visually corroborating these findings.

Finally, the Monte Carlo null model test over 1,000 random graphs confirmed that the observed modularity of 0.3673 is significantly higher than the null distribution ($p = 0$), ruling out chance as an explanation for the community structure.

5 Conclusion

This project explored the structure of the Wikispeedia hyperlink network through visualization, centrality analysis, community detection, adjacency matrix examination, and statistical testing. The network exhibits properties common to many real-world information networks: it is sparse, has a heavy-tailed degree distribution, shows small-world characteristics, and contains well-defined communities that correspond to real topical categories.

One interesting direction for future work would be to incorporate the navigation path data that comes with the Wikispeedia dataset. By analyzing which paths players actually take, we could build a weighted network where edge weights reflect how frequently each link is clicked. This would let us study not just the static hyperlink structure, but the dynamic patterns of how humans actually navigate through the network. Another extension would be to compare different null models—for example, using a configuration model that preserves the degree sequence rather than just the number of edges, which would provide a stricter test of community significance.

6 References

1. West, R. and Leskovec, J. (2012). Human Wayfinding in Information Networks. *21st International World Wide Web Conference (WWW).*

2. West, R., Pineau, J., and Precup, D. (2009). Wikispeedia: An Online Game for Inferring Semantic Distances between Concepts. *21st International Joint Conference on Artificial Intelligence (IJCAI).*

3. SNAP Dataset: Wikispeedia navigation paths. Stanford Network Analysis Project. https://snap.stanford.edu/data/wikispeedia.html

4. Csardi, G. and Nepusz, T. (2006). The igraph software package for complex network research. *InterJournal, Complex Systems*, 1695.

7 Appendix: Code

All R code used in this analysis is reproduced below.

```
knitr::opts_chunk$set(echo = FALSE, warning = FALSE, message = FALSE,
                      fig.width = 6, fig.height = 4.5, fig.align = "center",
                      out.width = "100%")
library(igraph)
library(dplyr)
library(Matrix)
set.seed(194)
# Read the edge list (skip comment lines starting with #)
edges <- read.delim("wikispeedia_paths-and-graph/links.tsv",
                    header = FALSE, comment.char = "#",
                    stringsAsFactors = FALSE)
colnames(edges) <- c("from", "to")

# Build directed igraph
wiki_net <- graph_from_data_frame(edges, directed = TRUE)

# Read categories and attach top-level category as vertex attribute
cats <- read.delim("wikispeedia_paths-and-graph/categories.tsv",
                   header = FALSE, comment.char = "#",
                   stringsAsFactors = FALSE)
colnames(cats) <- c("article", "category")

# Extract top-level category (e.g., "subject.Science.Physics" -> "Science")
cats <- cats %>%
  mutate(top_cat = sapply(strsplit(category, "\\."), function(x) {
    if (length(x) >= 2) x[2] else "Other"
  }))

# Keep first category per article
cats_unique <- cats %>% distinct(article, .keep_all = TRUE)

# Assign to vertices
V(wiki_net)$category <- cats_unique$top_cat[match(V(wiki_net)$name,
                                                  cats_unique$article)]
V(wiki_net)$category[is.na(V(wiki_net)$category)] <- "Uncategorized"
cat("The full network contains", vcount(wiki_net), "nodes and",
    ecount(wiki_net), "directed edges, with a density of",
    round(graph.density(wiki_net), 5), ".\n")
set.seed(194)
sub1 <- induced_subgraph(wiki_net, sample(V(wiki_net), 200))
sub2 <- induced_subgraph(wiki_net, sample(V(wiki_net), 200))
sub3 <- induced_subgraph(wiki_net, sample(V(wiki_net), 200))
sub4 <- induced_subgraph(wiki_net, sample(V(wiki_net), 200))

subgraph_summary <- data.frame(
  Subgraph = paste0("sub", 1:4),
  Nodes = c(vcount(sub1), vcount(sub2), vcount(sub3), vcount(sub4)),
  Edges = c(ecount(sub1), ecount(sub2), ecount(sub3), ecount(sub4)),
  Density = round(c(graph.density(sub1), graph.density(sub2),
                    graph.density(sub3), graph.density(sub4)), 4)
```

```r
)
knitr::kable(subgraph_summary, caption = "Summary of four induced subgraphs (200 nodes each)")
# Create a consistent color palette for categories
all_cats <- sort(unique(V(wiki_net)$category))
cat_colors <- rainbow(length(all_cats), alpha = 0.8)
names(cat_colors) <- all_cats
par(mfrow = c(1, 3), mar = c(1, 1, 2, 1))
node_col1 <- cat_colors[V(sub1)$category]
node_size1 <- 2 + 1.5 * sqrt(degree(sub1))

plot(sub1, layout = layout_with_fr(sub1),
     vertex.color = node_col1, vertex.size = node_size1,
     vertex.label = NA, edge.arrow.size = 0.2,
     edge.color = rgb(0.3, 0.3, 0.3, 0.3),
     main = "Subgraph 1 - Fruchterman-Reingold")

plot(sub1, layout = layout_with_kk(sub1),
     vertex.color = node_col1, vertex.size = node_size1,
     vertex.label = NA, edge.arrow.size = 0.2,
     edge.color = rgb(0.3, 0.3, 0.3, 0.3),
     main = "Subgraph 1 - Kamada-Kawai")

plot(sub1, layout = layout_in_circle(sub1),
     vertex.color = node_col1, vertex.size = node_size1,
     vertex.label = NA, edge.arrow.size = 0.2,
     edge.color = rgb(0.3, 0.3, 0.3, 0.3),
     main = "Subgraph 1 - Circular")

# Inset legend on last panel
legend("bottomleft", legend = all_cats, fill = cat_colors,
       cex = 0.35, ncol = 2, bty = "n", title = "Category")
par(mfrow = c(1, 3), mar = c(1, 1, 2, 1))
node_col2 <- cat_colors[V(sub2)$category]
node_size2 <- 2 + 1.5 * sqrt(degree(sub2))

plot(sub2, layout = layout_with_fr(sub2),
     vertex.color = node_col2, vertex.size = node_size2,
     vertex.label = NA, edge.arrow.size = 0.2,
     edge.color = rgb(0.3, 0.3, 0.3, 0.3),
     main = "Subgraph 2 - Fruchterman-Reingold")

plot(sub2, layout = layout_with_kk(sub2),
     vertex.color = node_col2, vertex.size = node_size2,
     vertex.label = NA, edge.arrow.size = 0.2,
     edge.color = rgb(0.3, 0.3, 0.3, 0.3),
     main = "Subgraph 2 - Kamada-Kawai")

plot(sub2, layout = layout_in_circle(sub2),
     vertex.color = node_col2, vertex.size = node_size2,
     vertex.label = NA, edge.arrow.size = 0.2,
     edge.color = rgb(0.3, 0.3, 0.3, 0.3),
     main = "Subgraph 2 - Circular")
par(mfrow = c(1, 3), mar = c(1, 1, 2, 1))
```

```r
node_col3 <- cat_colors[V(sub3)$category]
node_size3 <- 2 + 1.5 * sqrt(degree(sub3))

plot(sub3, layout = layout_with_fr(sub3),
     vertex.color = node_col3, vertex.size = node_size3,
     vertex.label = NA, edge.arrow.size = 0.2,
     edge.color = rgb(0.3, 0.3, 0.3, 0.3),
     main = "Subgraph 3 - Fruchterman-Reingold")

plot(sub3, layout = layout_with_kk(sub3),
     vertex.color = node_col3, vertex.size = node_size3,
     vertex.label = NA, edge.arrow.size = 0.2,
     edge.color = rgb(0.3, 0.3, 0.3, 0.3),
     main = "Subgraph 3 - Kamada-Kawai")

plot(sub3, layout = layout_in_circle(sub3),
     vertex.color = node_col3, vertex.size = node_size3,
     vertex.label = NA, edge.arrow.size = 0.2,
     edge.color = rgb(0.3, 0.3, 0.3, 0.3),
     main = "Subgraph 3 - Circular")
par(mfrow = c(1, 3), mar = c(1, 1, 2, 1))
node_col4 <- cat_colors[V(sub4)$category]
node_size4 <- 2 + 1.5 * sqrt(degree(sub4))

plot(sub4, layout = layout_with_fr(sub4),
     vertex.color = node_col4, vertex.size = node_size4,
     vertex.label = NA, edge.arrow.size = 0.2,
     edge.color = rgb(0.3, 0.3, 0.3, 0.3),
     main = "Subgraph 4 - Fruchterman-Reingold")

plot(sub4, layout = layout_with_kk(sub4),
     vertex.color = node_col4, vertex.size = node_size4,
     vertex.label = NA, edge.arrow.size = 0.2,
     edge.color = rgb(0.3, 0.3, 0.3, 0.3),
     main = "Subgraph 4 - Kamada-Kawai")

plot(sub4, layout = layout_in_circle(sub4),
     vertex.color = node_col4, vertex.size = node_size4,
     vertex.label = NA, edge.arrow.size = 0.2,
     edge.color = rgb(0.3, 0.3, 0.3, 0.3),
     main = "Subgraph 4 - Circular")
# Reciprocity
recip <- reciprocity(wiki_net)

# Transitivity (global clustering coefficient)
trans <- transitivity(wiki_net, type = "global")

# Degree assortativity
assort <- assortativity_degree(wiki_net, directed = TRUE)

# For diameter and avg path length, use largest strongly connected component
comp <- components(wiki_net, mode = "strong")
lcc_ids <- which(comp$membership == which.max(comp$csize))
```

```r
wiki_lcc <- induced_subgraph(wiki_net, lcc_ids)

diam <- diameter(wiki_lcc, directed = TRUE)
avg_path <- mean_distance(wiki_lcc, directed = TRUE)

metrics_df <- data.frame(
  Metric = c("Nodes", "Edges", "Density", "Reciprocity",
             "Global Clustering Coeff.", "Degree Assortativity",
             "Diameter (largest SCC)", "Avg. Path Length (largest SCC)"),
  Value = c(vcount(wiki_net), ecount(wiki_net),
            round(graph.density(wiki_net), 5),
            round(recip, 4), round(trans, 4), round(assort, 4),
            diam, round(avg_path, 3))
)
knitr::kable(metrics_df, caption = "Whole-network metrics for the Wikispeedia hyperlink graph")
# In-degree and out-degree for full network
in_deg <- degree(wiki_net, mode = "in")
out_deg <- degree(wiki_net, mode = "out")

# Top 10 articles by in-degree (most linked-to)
top_in <- in_deg %>% sort(decreasing = TRUE) %>% head(10)
cat("Top 10 articles by in-degree (most linked-to):\n")
data.frame(Article = names(top_in), In_Degree = as.integer(top_in))
# Betweenness centrality on full network
btw <- betweenness(wiki_net, directed = TRUE)
top_btw <- btw %>% sort(decreasing = TRUE) %>% head(10)
cat("Top 10 articles by betweenness centrality:\n")
data.frame(Article = names(top_btw),
           Betweenness = round(as.numeric(top_btw), 1))
# Eigenvector centrality
eigen_vals <- eigen_centrality(wiki_net, directed = TRUE)$vector %>%
  sort(decreasing = TRUE)
top_eigen <- eigen_vals %>% head(10)
cat("Top 10 articles by eigenvector centrality:\n")
data.frame(Article = names(top_eigen),
           Eigen_Centrality = round(as.numeric(top_eigen), 3))
par(mfrow = c(1, 2))

# In-degree distribution (log-log)
in_tab <- in_deg %>% table()
plot(as.numeric(names(in_tab)), as.numeric(in_tab),
     log = "xy", pch = 16, cex = 0.7, col = "steelblue",
     xlab = "In-Degree (k)", ylab = "Frequency",
     main = "In-Degree Distribution (log-log)")

# Out-degree distribution (log-log)
out_tab <- out_deg %>% table()
plot(as.numeric(names(out_tab)), as.numeric(out_tab),
     log = "xy", pch = 16, cex = 0.7, col = "tomato",
     xlab = "Out-Degree (k)", ylab = "Frequency",
     main = "Out-Degree Distribution (log-log)")
ebtw <- edge_betweenness(wiki_net, directed = TRUE)
cat("Edge betweenness summary:\n")
```

```r
summary(ebtw)
cat("\nTop 10 edges by betweenness:\n")
top_edges_idx <- ebtw %>% order(decreasing = TRUE) %>% head(10)
top_edge_list <- as_edgelist(wiki_net)[top_edges_idx, ]
data.frame(From = top_edge_list[, 1],
           To = top_edge_list[, 2],
           Edge_Betweenness = round(ebtw[top_edges_idx], 1))
# Convert subgraph 1 to undirected for community detection
sub1_und <- as.undirected(sub1, mode = "collapse")

# Algorithm 1: Fast Greedy (modularity optimization)
fg <- cluster_fast_greedy(sub1_und)

# Algorithm 2: Louvain
lv <- cluster_louvain(sub1_und)

# Algorithm 3: Walktrap (random walk based)
wt <- cluster_walktrap(sub1_und)

comm_compare <- data.frame(
  Algorithm = c("Fast Greedy", "Louvain", "Walktrap"),
  Communities = c(length(fg), length(lv), length(wt)),
  Modularity = round(c(modularity(fg), modularity(lv), modularity(wt)), 4)
)
knitr::kable(comm_compare,
             caption = "Comparison of community detection algorithms on Subgraph 1")
par(mfrow = c(1, 3), mar = c(1, 1, 2, 1))

plot(fg, sub1_und, vertex.size = 5, vertex.label = NA,
     edge.color = rgb(0.3, 0.3, 0.3, 0.3),
     main = paste0("Fast Greedy (Q=", round(modularity(fg), 3), ")"))

plot(lv, sub1_und, vertex.size = 5, vertex.label = NA,
     edge.color = rgb(0.3, 0.3, 0.3, 0.3),
     main = paste0("Louvain (Q=", round(modularity(lv), 3), ")"))

plot(wt, sub1_und, vertex.size = 5, vertex.label = NA,
     edge.color = rgb(0.3, 0.3, 0.3, 0.3),
     main = paste0("Walktrap (Q=", round(modularity(wt), 3), ")"))
# Use the algorithm with highest modularity
mod_scores <- c(modularity(fg), modularity(lv), modularity(wt))
best_idx <- which.max(mod_scores)
best <- list(fg, lv, wt)[[best_idx]]
best_name <- c("Fast Greedy", "Louvain", "Walktrap")[best_idx]

cat("Best algorithm:", best_name, "\n\n")
cat("Category composition of top communities:\n\n")

mem <- membership(best)
top_comms <- names(sort(table(mem), decreasing = TRUE))[1:min(5, length(unique(mem)))]

for (cc in top_comms) {
  articles_in_comm <- V(sub1_und)$name[mem == as.integer(cc)]
```

```r
  comm_cats <- V(sub1)$category[V(sub1)$name %in% articles_in_comm]
  cat(paste0("Community ", cc, " (", length(articles_in_comm), " articles): "))
  top_cats <- sort(table(comm_cats), decreasing = TRUE)[1:min(3, length(unique(comm_cats)))]
  cat(paste(names(top_cats), paste0("(", top_cats, ")"), collapse = ", "), "\n")
}
# Run Louvain on all four subgraphs for comparison
sub_und_list <- lapply(list(sub1, sub2, sub3, sub4), as.undirected, mode = "collapse")
lv_all <- lapply(sub_und_list, cluster_louvain)

cross_sub <- data.frame(
  Subgraph = paste0("sub", 1:4),
  Communities = sapply(lv_all, length),
  Modularity = sapply(lv_all, modularity) %>% round(4)
)
knitr::kable(cross_sub,
             caption = "Louvain community detection across all four subgraphs")
par(mfrow = c(1, 2), mar = c(2, 2, 3, 1))

A1 <- as_adjacency_matrix(sub1, sparse = TRUE)

# Original ordering
image(A1, main = "Adjacency Matrix - Original Order",
      xlab = "", ylab = "", sub = "")

# Reordered by community membership
ord <- order(membership(best))
image(A1[ord, ord], main = paste0("Adjacency Matrix - Reordered by ", best_name),
      xlab = "", ylab = "", sub = "")
# Run Louvain on full undirected network for observed modularity
wiki_und <- as.undirected(wiki_net, mode = "collapse")
lv_full <- cluster_louvain(wiki_und)
Q_obs <- modularity(lv_full)

# Monte Carlo null model simulation
n_sim <- 1000
Q_null <- numeric(n_sim)

n_nodes_full <- vcount(wiki_und)
n_edges_full <- ecount(wiki_und)

for (i in 1:n_sim) {
  g_rand <- erdos.renyi.game(n_nodes_full, n_edges_full, type = "gnm")
  lv_rand <- cluster_louvain(g_rand)
  Q_null[i] <- modularity(lv_rand)
}

# p-value: fraction of null Q >= observed Q
p_val <- mean(Q_null >= Q_obs)

# Compute histogram bins from null data only, then extend x-axis to show Q_obs
h <- hist(Q_null, breaks = 30, plot = FALSE)
plot(h, col = "lightblue", border = "grey",
     main = "Monte Carlo Null Model: Modularity (Full Network)",
```

```r
     xlab = "Modularity (Q)",
     xlim = c(min(h$breaks), max(Q_obs + 0.02, max(h$breaks))))
abline(v = Q_obs, col = "red", lwd = 2.5, lty = 2)
legend("topright",
       legend = c(paste0("Observed Q = ", round(Q_obs, 4)),
                  paste0("p-value = ", p_val)),
       col = c("red", NA), lwd = c(2, NA), lty = c(2, NA),
       bty = "n", cex = 0.9)
```